In [1]:
```
# This cell is added by sphinx-gallery
%matplotlib inline
%pip install pennylane torch torchvision
```

```
Requirement already satisfied: pennylane in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (0.44.1)
Requirement already satisfied: torch in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (2.11.0)
Requirement already satisfied: torchvision in /home/msoliviawatt/machine_lea
rning/ml-env/lib/python3.13/site-packages (0.26.0)
Requirement already satisfied: scipy in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (from pennylane) (1.17.1)
Requirement already satisfied: networkx in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (3.6.1)
Requirement already satisfied: rustworkx>=0.14.0 in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.17.1)
Requirement already satisfied: autograd in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (1.8.0)
Requirement already satisfied: appdirs in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from pennylane) (1.4.4)
Requirement already satisfied: autoray==0.8.2 in /home/msoliviawatt/machine_
learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.8.2)
Requirement already satisfied: cachetools in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from pennylane) (7.0.5)
Requirement already satisfied: pennylane-lightning>=0.44 in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from pennylane) (0.
44.0)
Requirement already satisfied: requests in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from pennylane) (2.32.5)
Requirement already satisfied: tomlkit in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from pennylane) (0.14.0)
Requirement already satisfied: typing_extensions in /home/msoliviawatt/machi
ne_learning/ml-env/lib/python3.13/site-packages (from pennylane) (4.15.0)
Requirement already satisfied: packaging in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (from pennylane) (26.0)
Requirement already satisfied: diastatic-malt==2.15.2 in /home/msoliviawatt/
machine_learning/ml-env/lib/python3.13/site-packages (from pennylane) (2.15.
2)
Requirement already satisfied: gast in /home/msoliviawatt/machine_learning/m
l-env/lib/python3.13/site-packages (from pennylane) (0.7.0)
Requirement already satisfied: numpy in /home/msoliviawatt/machine_learning/
ml-env/lib/python3.13/site-packages (from pennylane) (2.4.2)
Requirement already satisfied: astunparse in /home/msoliviawatt/machine_lear
ning/ml-env/lib/python3.13/site-packages (from diastatic-malt==2.15.2->penny
lane) (1.6.3)
Requirement already satisfied: termcolor in /home/msoliviawatt/machine_learn
ing/ml-env/lib/python3.13/site-packages (from diastatic-malt==2.15.2->pennyl
ane) (3.3.0)
Requirement already satisfied: filelock in /home/msoliviawatt/machine_learni
ng/ml-env/lib/python3.13/site-packages (from torch) (3.25.2)
Requirement already satisfied: setuptools<82 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (81.0.0)
Requirement already satisfied: sympy>=1.13.3 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (1.14.0)
Requirement already satisfied: jinja2 in /home/msoliviawatt/machine_learnin
g/ml-env/lib/python3.13/site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (2026.3.0)
Requirement already satisfied: cuda-toolkit==13.0.2 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[cubla
```

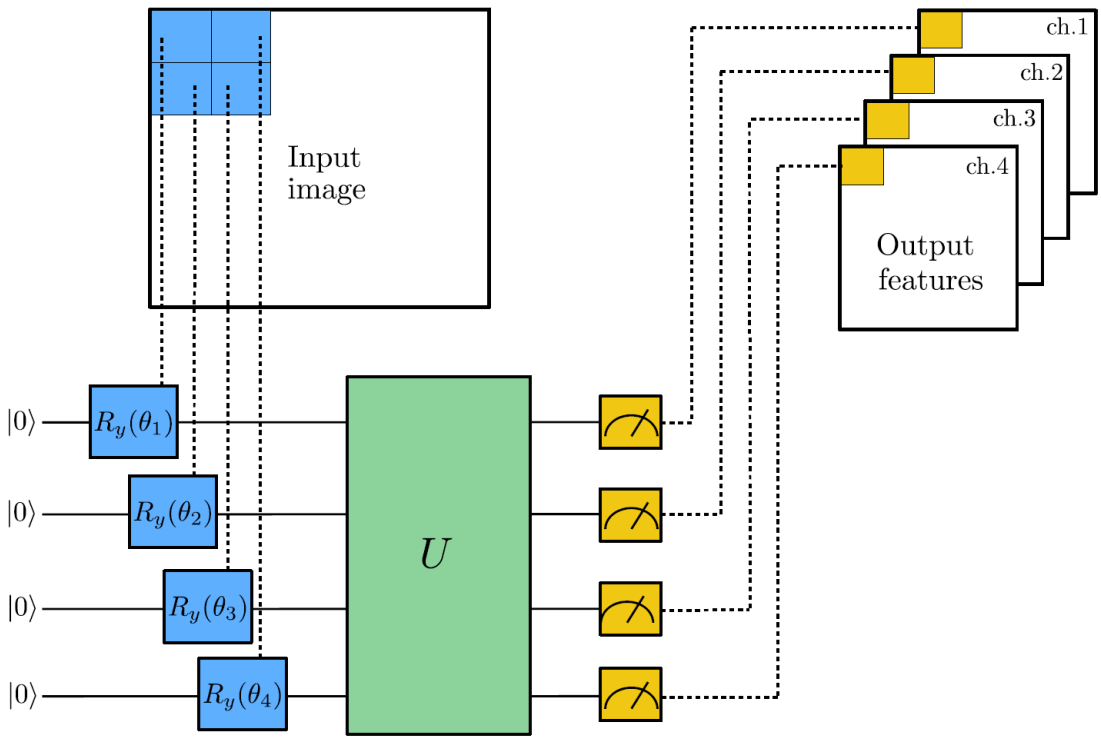
```
s,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]==
13.0.2; platform_system == "Linux"->torch) (13.0.2)
Requirement already satisfied: cuda-bindings<14,>=13.0.3 in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (13.2.
0)
Requirement already satisfied: nvidia-cudnn-cu13==9.19.0.56 in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (9.1
9.0.56)
Requirement already satisfied: nvidia-cusparselt-cu13==0.8.0 in /home/msoliv
iawatt/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (0.
8.0)
Requirement already satisfied: nvidia-nccl-cu13==2.28.9 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (2.28.9)
Requirement already satisfied: nvidia-nvshmem-cu13==3.4.5 in /home/msoliviaw
att/machine_learning/ml-env/lib/python3.13/site-packages (from torch) (3.4.
5)
Requirement already satisfied: triton==3.6.0 in /home/msoliviawatt/machine_l
earning/ml-env/lib/python3.13/site-packages (from torch) (3.6.0)
Requirement already satisfied: nvidia-cublas==13.1.0.3.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (13.1.0.3)
Requirement already satisfied: nvidia-cuda-runtime==13.0.96.* in /home/msoli
viawatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-tool
kit[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrt
c,nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.96)
Requirement already satisfied: nvidia-cufft==12.0.0.61.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (12.0.0.61)
Requirement already satisfied: nvidia-cufile==1.15.1.6.* in /home/msoliviawa
tt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[c
ublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvt
x]==13.0.2; platform_system == "Linux"->torch) (1.15.1.6)
Requirement already satisfied: nvidia-cuda-cupti==13.0.85.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: nvidia-curand==10.4.0.35.* in /home/msoliviaw
att/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (10.4.0.35)
Requirement already satisfied: nvidia-cusolver==12.0.4.66.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (12.0.4.66)
Requirement already satisfied: nvidia-cusparse==12.6.3.3.* in /home/msolivia
watt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (12.6.3.3)
Requirement already satisfied: nvidia-nvjitlink==13.0.88.* in /home/msolivia
watt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit
[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,n
vtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-cuda-nvrtc==13.0.88.* in /home/msolivi
awatt/machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolki
```

```
t[cublas,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,
nvtx]==13.0.2; platform_system == "Linux"->torch) (13.0.88)
Requirement already satisfied: nvidia-nvtx==13.0.85.* in /home/msoliviawatt/
machine_learning/ml-env/lib/python3.13/site-packages (from cuda-toolkit[cubl
as,cudart,cufft,cufile,cupti,curand,cusolver,cusparse,nvjitlink,nvrtc,nvtx]=
=13.0.2; platform_system == "Linux"->torch) (13.0.85)
Requirement already satisfied: cuda-pathfinder~=1.1 in /home/msoliviawatt/ma
chine_learning/ml-env/lib/python3.13/site-packages (from cuda-bindings<14,>=
13.0.3->torch) (1.5.1)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /home/msoliviawatt/m
achine_learning/ml-env/lib/python3.13/site-packages (from torchvision) (12.
1.1)
Requirement already satisfied: scipy-openblas32>=0.3.26 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from pennylane-light
ning>=0.44->pennylane) (0.3.31.188.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from sympy>=1.13.3->torch)
(1.3.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from astunparse->diastatic
-malt==2.15.2->pennylane) (0.46.3)
Requirement already satisfied: six<2.0,>=1.6.1 in /home/msoliviawatt/machine
_learning/ml-env/lib/python3.13/site-packages (from astunparse->diastatic-ma
lt==2.15.2->pennylane) (1.17.0)
Requirement already satisfied: MarkupSafe>=2.0 in /home/msoliviawatt/machine
_learning/ml-env/lib/python3.13/site-packages (from jinja2->torch) (3.0.3)
Requirement already satisfied: charset_normalizer<4,>=2 in /home/msoliviawat
t/machine_learning/ml-env/lib/python3.13/site-packages (from requests->penny
lane) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /home/msoliviawatt/machine_le
arning/ml-env/lib/python3.13/site-packages (from requests->pennylane) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests->pennylane)
(2.6.3)
Requirement already satisfied: certifi>=2017.4.17 in /home/msoliviawatt/mach
ine_learning/ml-env/lib/python3.13/site-packages (from requests->pennylane)
(2026.1.4)
Note: you may need to restart the kernel to use updated packages.
```

# Quanvolutional Neural Networks {#quanvolution}

> > Warning
>
> This demo uses TensorFlow, which is a deprecated interface with PennyLane v0.42. Interfacing with TensorFlow will no longer be supported with PennyLane v0.43 and higher. Consider switching to a different machine learning interface with PennyLane, like PyTorch or JAX.

In this demo we implement the *Quanvolutional Neural Network*, a quantum machine learning model originally introduced in Henderson et al. (2019).




# Introduction

## Classical convolution

The *convolutional neural network* (CNN) is a standard model in classical machine learning which is particularly suitable for processing images. The model is based on the idea of a *convolution layer* where, instead of processing the full input data with a global function, a local convolution is applied.

If the input is an image, small local regions are sequentially processed with the same kernel. The results obtained for each region are usually associated to different channels of a single output pixel. The union of all the output pixels produces a new image-like object, which can be further processed by additional layers.

## Quantum convolution

One can extend the same idea also to the context of quantum variational circuits. A possible approach is given by the following procedure which is very similar to the one used in Ref. [1]. The scheme is also represented in the figure at the top of this tutorial.

1. A small region of the input image, in our example a $2 \times 2$ square, is embedded into a quantum circuit. In this demo, this is achieved with parametrized rotations applied to the qubits initialized in the ground state.
2. A quantum computation, associated to a unitary $U$, is performed on the system. The unitary could be generated by a variational quantum circuit or, more simply, by a random circuit as proposed in Ref. [1].
3. The quantum system is finally measured, obtaining a list of classical expectation values. The measurement results could also be classically post-processed as proposed in Ref. [1] but, for simplicity, in this demo we directly use the raw expectation values.
4. Analogously to a classical convolution layer, each expectation value is mapped to a different channel of a single output pixel.
5. Iterating the same procedure over different regions, one can scan the full input image, producing an output object which will be structured as a multi-channel image.
6. The quantum convolution can be followed by further quantum layers or by classical layers.

The main difference with respect to a classical convolution is that a quantum circuit can generate highly complex kernels whose computation could be, at least in principle, classically intractable.

> > Note
>
> In this tutorial we follow the approach of Ref. [1] in which a fixed non-trainable quantum circuit is used as a "quanvolution" kernel, while the subsequent classical layers are trained for the classification problem of interest. However, by leveraging the ability of PennyLane to evaluate gradients of quantum circuits, the quantum kernel could also be trained.

# General setup

This Python code requires *PennyLane* with the *TensorFlow* interface and the plotting library *matplotlib*.

In [2]:
```
import pennylane as qml
from pennylane import numpy as np
from pennylane.templates import RandomLayers
import matplotlib.pyplot as plt

# removed tensorflow and added pytorch stuff
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
```

```
import os
```

# Setting of the main hyper-parameters of the model

In [3]:
```
NUM_EPOCHS = 50
NUM_LAYERS = 1
NUM_TRAIN = 200
NUM_TEST = 80
NUM_CLASSES = 10
LEARNING_RATE = 0.001
BATCH_SIZE = 32


SAVE_PATH = "./quanv_data/"
os.makedirs(SAVE_PATH, exist_ok=True)

PREPROCESS = True

np.random.seed(0)
# tf.random.set_seed(0)
```

# Loading of the Stanford Cars dataset

In [4]:
```
from torchvision.datasets import StanfordCars
from torchvision import transforms
from torch.utils.data import Dataset, Subset, random_split, DataLoader
import torch

square_size = 28

base_transform = transforms.Compose([
    transforms.Grayscale(num_output_channels=1),
    transforms.Resize((square_size, square_size)),
    transforms.ToTensor()
])

full_data = StanfordCars(
    root="./data",
    split="train",
    download=False,
    transform=base_transform
)

print(len(full_data.classes))
```

196

In [5]:
```
for i, name in enumerate(full_data.classes[:10]):
    print(i, name)
```

```
0 AM General Hummer SUV 2000
1 Acura RL Sedan 2012
2 Acura TL Sedan 2012
3 Acura TL Type-S 2008
4 Acura TSX Sedan 2012
5 Acura Integra Type R 2001
6 Acura ZDX Hatchback 2012
7 Aston Martin V8 Vantage Convertible 2012
8 Aston Martin V8 Vantage Coupe 2012
9 Aston Martin Virage Convertible 2012
```

In [6]:

```
selected_classes = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
NUM_CLASSES = len(selected_classes)
print(NUM_CLASSES)
```

```
10
```

In [7]:

```
# manually splitting the data
selected_indices = [
    i for i, (_, label) in enumerate(full_data._samples)
    if label in selected_classes
]

filtered_data = Subset(full_data, selected_indices)

class_map = {old_label: new_label for new_label, old_label in enumerate(sele

class RemappedSubset(Dataset):
    def __init__(self, subset, class_map):
        self.subset = subset
        self.class_map = class_map

    def __len__(self):
        return len(self.subset)

    def __getitem__(self, idx):
        image, label = self.subset[idx]
        return image, self.class_map[label]

remapped_data = RemappedSubset(filtered_data, class_map)

total_size = len(remapped_data)
train_size = int(0.8 * total_size)
val_size = total_size - train_size

generator = torch.Generator().manual_seed(42)

train_subset, val_subset = random_split(
    remapped_data,
    [train_size, val_size],
    generator=generator
)

train_dl = DataLoader(train_subset, batch_size=16, shuffle=True)
val_dl = DataLoader(val_subset, batch_size=16, shuffle=False)
```

```
print("Total filtered samples:", len(remapped_data))
print("Train samples:", len(train_subset))
print("Val samples:", len(val_subset))
print("Class map:", class_map)
```

```
Total filtered samples: 406
Train samples: 324
Val samples: 82
Class map: {0: 0, 1: 1, 2: 2, 3: 3, 4: 4, 5: 5, 6: 6, 7: 7, 8: 8, 9: 9}
```

# Quantum circuit as a convolution kernel

We follow the scheme described in the introduction and represented in the figure at the top of this demo.

We initialize a PennyLane `default.qubit` device, simulating a system of $4$ qubits. The associated `qnode` represents the quantum circuit consisting of:

1. an embedding layer of local $R_y$ rotations (with angles scaled by a factor of $\pi$ );
2. a random circuit of `NUM_LAYERS` ;
3. a final measurement in the computational basis, estimating $4$ expectation values.

In [8]:
```
dev = qml.device("default.qubit", wires=4)
# Random circuit parameters
rand_params = np.random.uniform(high=2 * np.pi, size=(NUM_LAYERS, 4))

@qml.qnode(dev)
def circuit(phi):
    # Encoding of 4 classical input values
    for j in range(4):
        qml.RY(np.pi * phi[j], wires=j)

    # Random quantum circuit
    RandomLayers(rand_params, wires=list(range(4)))

    # Measurement producing 4 classical output values
    return [qml.expval(qml.PauliZ(j)) for j in range(4)]
```

The next function defines the convolution scheme:

1. the image is divided into squares of $2 \times 2$ pixels;
2. each square is processed by the quantum circuit;
3. the $4$ expectation values are mapped into $4$ different channels of a single output pixel.

> > Note

> This process halves the resolution of the input image. In the standard language of CNN, this would correspond to a convolution with a $2 \times 2$ *kernel* and a *stride* equal to $2$.

In [9]:
```
# def quanv(image):
#     """Convolves the input image with many applications of the same quantu
#     out = np.zeros((square_size // 2, square_size // 2, num_classes))

#     # Loop over the coordinates of the top-left pixel of 2X2 squares
#     for j in range(0, square_size, 2):
#         for k in range(0, square_size, 2):
#             # Process a squared 2x2 region of the image with a quantum cir
#             q_results = circuit(
#                 [
#                     image[j, k, 0],
#                     image[j, k + 1, 0],
#                     image[j + 1, k, 0],
#                     image[j + 1, k + 1, 0]
#                 ]
#             )
#             # Assign expectation values to different channels of the outpu
#             for c in range(num_classes):
#                 out[j // 2, k // 2, c] = q_results[c]
#     return out
```

In [10]:
```
def quanv(image):
    h, w, c = image.shape
    out = np.zeros((h // 2, w // 2, 4))

    for j in range(0, h - 1, 2):
        for k in range(0, w - 1, 2):
            q_results = circuit([
                image[j, k, 0],
                image[j, k + 1, 0],
                image[j + 1, k, 0],
                image[j + 1, k + 1, 0]
            ])

            for ch in range(4):
                out[j // 2, k // 2, ch] = q_results[ch]

    return out
```

In [11]:
```
# NUM_TRAIN = 200
# NUM_TEST = 80
train_subset_small = Subset(train_subset, range(min(NUM_TRAIN, len(train_sub
val_subset_small = Subset(val_subset, range(min(NUM_TEST, len(val_subset))))

def dataset_to_numpy(dataset):
    images = []
    labels = []

    for img, label in dataset:
        # img is [C, H, W], but PennyLane tutorial expects [H, W, C]
```

```
        img_np = img.permute(1, 2, 0).numpy()
        images.append(img_np)
        labels.append(label)

    images = np.array(images)
    labels = np.array(labels)

    return images, labels

train_images, train_labels = dataset_to_numpy(train_subset_small)
test_images, test_labels = dataset_to_numpy(val_subset_small)

print("train_images shape: ", train_images.shape)
print("train_labels shape: ", train_labels.shape)
print("test_images shape: ", test_images.shape)
print("test_labels shape: ", test_labels.shape)
print("Unique train labels: ", np.unique(train_labels))
print("Unique test labels: ", np.unique(test_labels))
```

```
train_images shape:  (200, 28, 28, 1)
train_labels shape:  (200,)
test_images shape:  (80, 28, 28, 1)
test_labels shape:  (80,)
Unique train labels:  [0 1 2 3 4 5 6 7 8 9]
Unique test labels:  [0 1 2 3 4 5 6 7 8 9]
```

# Quantum pre-processing of the dataset

Since we are not going to train the quantum convolution layer, it is more efficient to apply it as a "pre-processing" layer to all the images of our dataset. Later an entirely classical model will be directly trained and tested on the pre-processed dataset, avoiding unnecessary repetitions of quantum computations.

The pre-processed images will be saved in the folder `SAVE_PATH`. Once saved, they can be directly loaded by setting `PREPROCESS = False`, otherwise the quantum convolution is evaluated at each run of the code.

In [12]:
```
if PREPROCESS == True:
    q_train_images = []
    print("Quantum pre-processing of train images:")
    for idx, img in enumerate(train_images):
        print("{}/{}        ".format(idx + 1, NUM_TRAIN), end="\r")
        q_train_images.append(quanv(img))
    q_train_images = np.asarray(q_train_images)

    q_test_images = []
    print("\nQuantum pre-processing of test images:")
    for idx, img in enumerate(test_images):
        print("{}/{}        ".format(idx + 1, NUM_TEST), end="\r")
        q_test_images.append(quanv(img))
    q_test_images = np.asarray(q_test_images)
```

```
    # Save pre-processed images
    np.save(SAVE_PATH + "q_train_images.npy", q_train_images)
    np.save(SAVE_PATH + "q_test_images.npy", q_test_images)


# Load pre-processed images
q_train_images = np.load(SAVE_PATH + "q_train_images.npy")
q_test_images = np.load(SAVE_PATH + "q_test_images.npy")
```

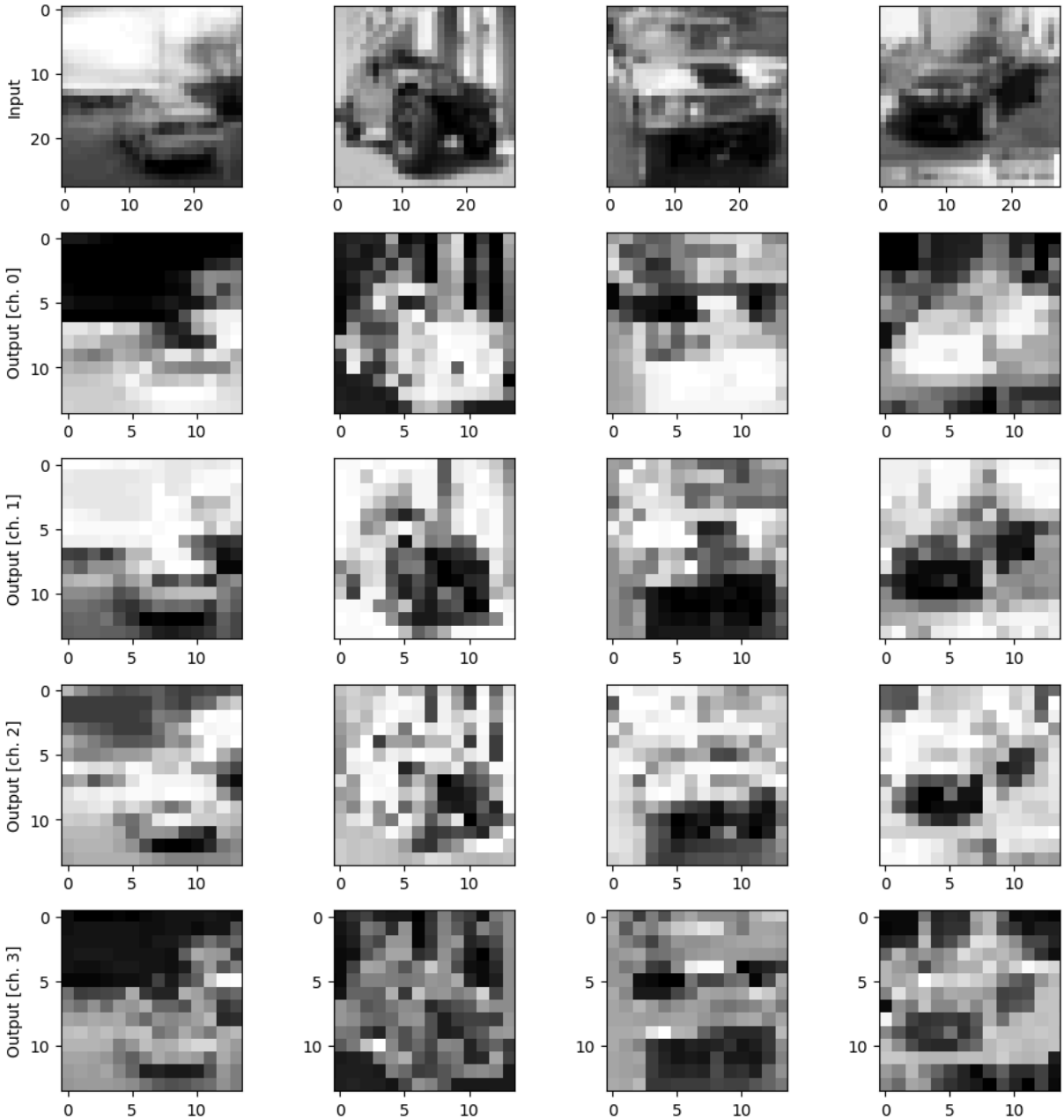
```
Quantum pre-processing of train images:
200/200
Quantum pre-processing of test images:
80/80
```

Let us visualize the effect of the quantum convolution layer on a batch of samples:

In [13]:
```
n_samples = 4
n_channels = 4
fig, axes = plt.subplots(1 + n_channels, n_samples, figsize=(10, 10))
for k in range(n_samples):
    axes[0, 0].set_ylabel("Input")
    if k != 0:
        axes[0, k].yaxis.set_visible(False)
    axes[0, k].imshow(train_images[k, :, :, 0], cmap="gray")

    # Plot all output channels
    for c in range(n_channels):
        axes[c + 1, 0].set_ylabel("Output [ch. {}]".format(c))
        if k != 0:
            axes[c, k].yaxis.set_visible(False)
        axes[c + 1, k].imshow(q_train_images[k, :, :, c], cmap="gray")

plt.tight_layout()
plt.show()
```




Below each input image, the $4$ output channels generated by the quantum convolution are visualized in gray scale.

One can clearly notice the downsampling of the resolution and some local distortion introduced by the quantum kernel. On the other hand the global shape of the image is preserved, as expected for a convolution layer.

# Hybrid quantum-classical model

After the application of the quantum convolution layer we feed the resulting features into a classical neural network that will be trained to classify the $10$ different digits of the MNIST dataset.

We use a very simple model: just a fully connected layer with 10 output nodes with a final *softmax* activation function.

The model is compiled with a *stochastic-gradient-descent* optimizer, and a *cross-entropy* loss function.

In [14]:
```
# def MyModel():
#     """Initializes and returns a custom Keras model
#     which is ready to be trained."""
#     model = keras.models.Sequential([
#         keras.layers.Flatten(),
#         keras.layers.Dense(num_classes, activation="softmax")
#     ])

#     model.compile(
#         optimizer='adam',
#         loss="sparse_categorical_crossentropy",
#         metrics=["accuracy"],
#     )
#     return model

class MyModel(nn.Module):
    def __init__(self, input_shape, num_classes=10):
        super().__init__()
        self.flatten = nn.Flatten()
        self.net = nn.Sequential(
            nn.Linear(input_shape, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.flatten(x)
        return self.net(x)
```

# Training

We first initialize an instance of the model, then we train and validate it with the dataset that has been already pre-processed by a quantum convolution.

In [15]:
```
def train_model(model, X, y, epochs=30, lr=0.001, batch_size=32):
    dataset = TensorDataset(torch.tensor(X, dtype=torch.float32),
                            torch.tensor(y, dtype=torch.long))
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    history = {"loss": [], "accuracy": []}

    for epoch in range(epochs):
        total_loss = 0
```

```
        correct = 0
        total = 0

        for xb, yb in loader:
            optimizer.zero_grad()
            outputs = model(xb)
            loss = criterion(outputs, yb)
            loss.backward()
            optimizer.step()

            total_loss += loss.item()

            preds = torch.argmax(outputs, dim=1)
            correct += (preds == yb).sum().item()
            total += yb.size(0)

        acc = correct / total
        history["loss"].append(total_loss)
        history["accuracy"].append(acc)

        print(f"Epoch {epoch+1}: Loss={total_loss:.4f}, Acc={acc:.4f}")

    return history
```

In [16]:

```
# q_model = MyModel()

# q_history = q_model.fit(
#     q_train_images,
#     train_labels,
#     validation_data=(q_test_images, test_labels),
#     batch_size=4,
#     epochs=NUM,
#     verbose=2,
# )

input_size = q_train_images.shape[1] * q_train_images.shape[2] * q_train_ima

q_model = MyModel(input_size)

q_history = train_model(q_model, q_train_images, train_labels, epochs = NUM_
```

```
Epoch 1: Loss=16.0642, Acc=0.1300
Epoch 2: Loss=14.4390, Acc=0.2450
Epoch 3: Loss=13.7018, Acc=0.3500
Epoch 4: Loss=12.9054, Acc=0.4150
Epoch 5: Loss=11.9328, Acc=0.4950
Epoch 6: Loss=11.1243, Acc=0.5450
Epoch 7: Loss=9.9340, Acc=0.5950
Epoch 8: Loss=9.3564, Acc=0.6250
Epoch 9: Loss=8.4878, Acc=0.7000
Epoch 10: Loss=7.8589, Acc=0.7550
Epoch 11: Loss=7.0861, Acc=0.8200
Epoch 12: Loss=6.5442, Acc=0.8550
Epoch 13: Loss=5.8778, Acc=0.9000
Epoch 14: Loss=5.2309, Acc=0.9150
Epoch 15: Loss=4.8564, Acc=0.9300
Epoch 16: Loss=4.2269, Acc=0.9450
Epoch 17: Loss=3.7650, Acc=0.9700
Epoch 18: Loss=3.4440, Acc=0.9750
Epoch 19: Loss=2.9031, Acc=0.9900
Epoch 20: Loss=2.6410, Acc=0.9950
Epoch 21: Loss=2.5330, Acc=0.9950
Epoch 22: Loss=2.1186, Acc=0.9950
Epoch 23: Loss=1.8807, Acc=0.9950
Epoch 24: Loss=1.8129, Acc=0.9950
Epoch 25: Loss=1.5105, Acc=1.0000
Epoch 26: Loss=1.3791, Acc=1.0000
Epoch 27: Loss=1.2502, Acc=1.0000
Epoch 28: Loss=1.0566, Acc=1.0000
Epoch 29: Loss=0.9433, Acc=1.0000
Epoch 30: Loss=0.9286, Acc=1.0000
Epoch 31: Loss=0.8205, Acc=1.0000
Epoch 32: Loss=0.7910, Acc=1.0000
Epoch 33: Loss=0.7031, Acc=1.0000
Epoch 34: Loss=0.6152, Acc=1.0000
Epoch 35: Loss=0.5460, Acc=1.0000
Epoch 36: Loss=0.5244, Acc=1.0000
Epoch 37: Loss=0.4818, Acc=1.0000
Epoch 38: Loss=0.4782, Acc=1.0000
Epoch 39: Loss=0.4336, Acc=1.0000
Epoch 40: Loss=0.4095, Acc=1.0000
Epoch 41: Loss=0.3886, Acc=1.0000
Epoch 42: Loss=0.3618, Acc=1.0000
Epoch 43: Loss=0.3459, Acc=1.0000
Epoch 44: Loss=0.3218, Acc=1.0000
Epoch 45: Loss=0.3101, Acc=1.0000
Epoch 46: Loss=0.2814, Acc=1.0000
Epoch 47: Loss=0.2741, Acc=1.0000
Epoch 48: Loss=0.2523, Acc=1.0000
Epoch 49: Loss=0.2455, Acc=1.0000
Epoch 50: Loss=0.2346, Acc=1.0000
```

In order to compare the results achievable with and without the quantum convolution layer, we initialize also a "classical" instance of the model that will be directly trained and validated with the raw MNIST images (i.e., without quantum pre-processing).

In [22]:
```
# c_model = MyModel()

# c_history = c_model.fit(
#     train_images,
#     train_labels,
#     validation_data=(test_images, test_labels),
#     batch_size=4,
#     epochs=NUM_EPOCHS,
#     verbose=2,
# )

input_size = train_images.shape[1] * train_images.shape[2] * train_images.sh

c_model = MyModel(input_size)

c_history = train_model(c_model, train_images, train_labels, epochs = NUM_EP
```

```
Epoch 1: Loss=16.1095, Acc=0.1350
Epoch 2: Loss=15.8353, Acc=0.1450
Epoch 3: Loss=15.8290, Acc=0.1650
Epoch 4: Loss=15.4709, Acc=0.2300
Epoch 5: Loss=15.4044, Acc=0.1550
Epoch 6: Loss=15.0757, Acc=0.1700
Epoch 7: Loss=14.9910, Acc=0.2800
Epoch 8: Loss=14.9806, Acc=0.2800
Epoch 9: Loss=14.9380, Acc=0.2450
Epoch 10: Loss=14.5987, Acc=0.2400
Epoch 11: Loss=14.5525, Acc=0.2500
Epoch 12: Loss=14.2630, Acc=0.3350
Epoch 13: Loss=13.9887, Acc=0.3650
Epoch 14: Loss=14.1528, Acc=0.3200
Epoch 15: Loss=13.5241, Acc=0.3500
Epoch 16: Loss=13.6656, Acc=0.3050
Epoch 17: Loss=13.5192, Acc=0.3450
Epoch 18: Loss=13.4421, Acc=0.3400
Epoch 19: Loss=13.3667, Acc=0.3500
Epoch 20: Loss=13.2504, Acc=0.3600
Epoch 21: Loss=12.8014, Acc=0.4000
Epoch 22: Loss=12.7657, Acc=0.4250
Epoch 23: Loss=12.5464, Acc=0.4700
Epoch 24: Loss=12.3864, Acc=0.4050
Epoch 25: Loss=12.1042, Acc=0.4600
Epoch 26: Loss=11.9096, Acc=0.4500
Epoch 27: Loss=11.8313, Acc=0.4100
Epoch 28: Loss=11.7210, Acc=0.4700
Epoch 29: Loss=11.7770, Acc=0.4500
Epoch 30: Loss=11.3581, Acc=0.4950
Epoch 31: Loss=11.3226, Acc=0.4800
Epoch 32: Loss=11.2489, Acc=0.5000
Epoch 33: Loss=11.1570, Acc=0.5300
Epoch 34: Loss=10.9642, Acc=0.5250
Epoch 35: Loss=10.6194, Acc=0.5350
Epoch 36: Loss=10.4273, Acc=0.5300
Epoch 37: Loss=10.5207, Acc=0.5650
Epoch 38: Loss=10.2863, Acc=0.5650
Epoch 39: Loss=10.0531, Acc=0.5250
Epoch 40: Loss=9.9415, Acc=0.5300
Epoch 41: Loss=10.0274, Acc=0.5650
Epoch 42: Loss=9.8890, Acc=0.5450
Epoch 43: Loss=9.5541, Acc=0.6250
Epoch 44: Loss=9.3047, Acc=0.6000
Epoch 45: Loss=9.5997, Acc=0.6000
Epoch 46: Loss=9.2125, Acc=0.6300
Epoch 47: Loss=9.1469, Acc=0.6400
Epoch 48: Loss=8.9519, Acc=0.6650
Epoch 49: Loss=8.9067, Acc=0.6100
Epoch 50: Loss=8.6487, Acc=0.6450
```

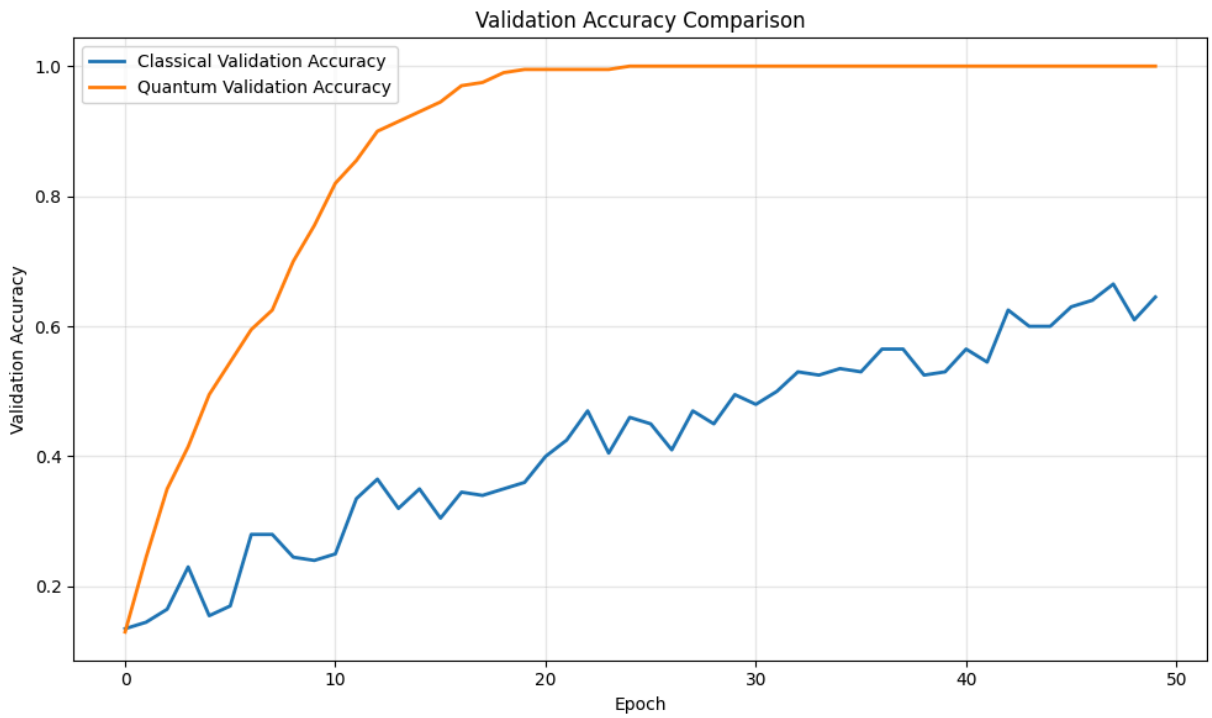
In [23]:

```
plt.figure(figsize=(10, 6))
plt.plot(c_history["accuracy"], label="Classical Validation Accuracy", linew
plt.plot(q_history["accuracy"], label="Quantum Validation Accuracy", linewic

plt.title("Validation Accuracy Comparison")
```

```
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# plt.plot(q_history["accuracy"], label="Quantum")
# plt.plot(c_history["accuracy"], label="Classical")
# plt.legend()
```

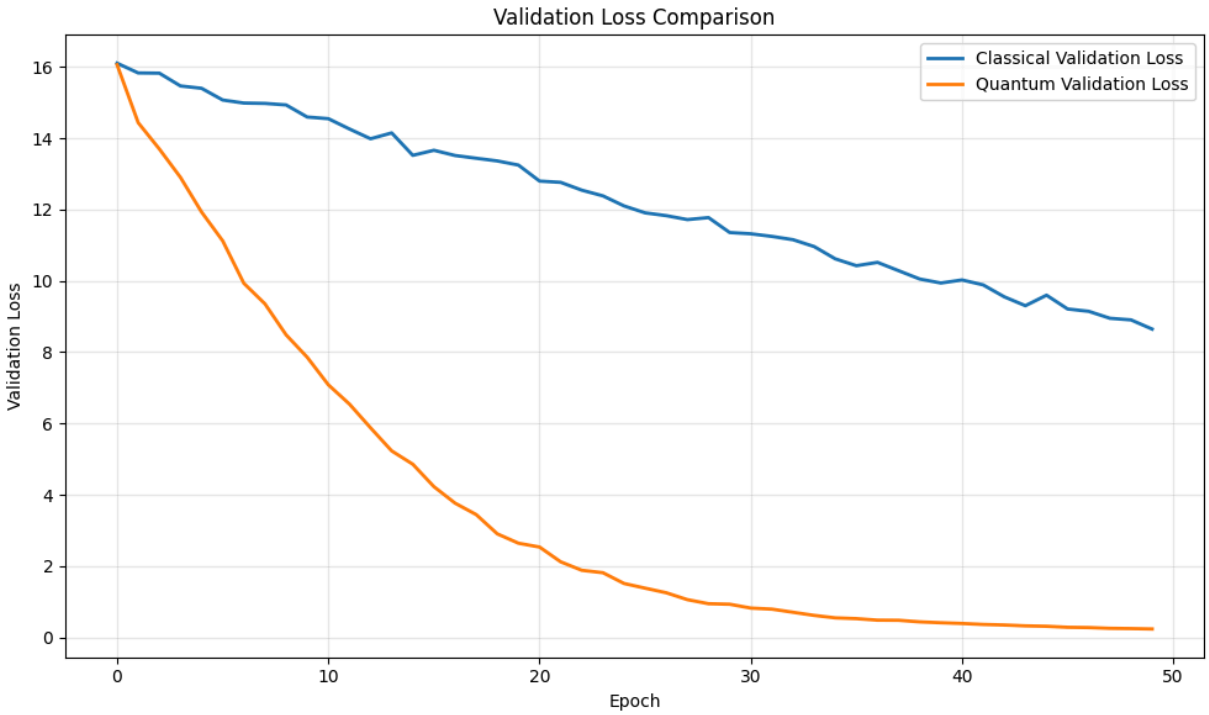



# Results

We can finally plot the test accuracy and the test loss with respect to the number of training epochs.

In [24]:
```
plt.figure(figsize=(10, 6))
plt.plot(c_history["loss"], label="Classical Validation Loss", linewidth=2)
plt.plot(q_history["loss"], label="Quantum Validation Loss", linewidth=2)

plt.title("Validation Loss Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Loss")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




1. Maxwell Henderson, Samriddhi Shakya, Shashindra Pradhan, Tristan Cook. "Quanvolutional Neural Networks: Powering Image Recognition with Quantum Circuits." arXiv:1904.04767, 2019.